

Introduction

Ana-Maria Paraschivoiu

Hadoop Assignment

This assignment explores a range of core technologies used in modern big data environments, including YARN, Hive, Impala, and Kafka. It covers key concepts such as resource management, distributed data storage, query optimisation, metadata management, and real-time data streaming. Through a combination of theoretical questions and practical examples, the assignment demonstrates how these technologies work together to process, manage, and analyse large volumes of data efficiently. Completing these exercises has helped develop both conceptual understanding and practical skills that are essential for working with Hadoop-based data platforms and distributed data processing systems

1. What are the two main sub-components of the Resource Manager and what does each do?

The ResourceManager has two main sub-components: the **Scheduler** and the **ApplicationsManager**. The **Scheduler** allocates cluster resources such as CPU and memory to applications based on scheduling policies. The **ApplicationsManager** accepts application submissions, starts the ApplicationMaster, and manages the lifecycle of each application until it completes.

2. Explain the difference between a container and an application master

A **container** is a package of computing resources, such as CPU and memory, that YARN provides to run a specific task. Containers perform the actual processing of data.

An **ApplicationMaster** is responsible for managing a single application. It communicates with the ResourceManager to request containers, schedules tasks to run inside those containers, monitors their progress, and handles failures if they occur.

In simple terms:

- **ApplicationMaster = Manager** (organises the work)
- **Container = Worker** (does the work)

3. Why does yarn launch a separate AM for every application instead of having the RM track job state directly?

YARN creates a separate **ApplicationMaster** for every application so that each job can manage itself. The ApplicationMaster requests containers, starts tasks, and monitors their progress. The **ResourceManager** only allocates resources and does not keep track of every task in every job. This makes YARN faster, more scalable, and easier to manage when many applications are running at the same time.

4. Compare Capacity Scheduler and Fair Scheduler – when would you choose one over the other?

Capacity Scheduler divides cluster resources into fixed queues, where each queue is assigned a guaranteed percentage of the available resources. It is best suited for large organisations with multiple departments because each team is guaranteed its allocated resources, ensuring predictable performance and preventing one department from using all of the cluster's capacity.

Fair Scheduler shares cluster resources equally among all running applications. If only one application is running, it can use almost all of the cluster resources. As more applications are submitted, the scheduler redistributes resources fairly so that each application receives a reasonable share. This is ideal for environments where users submit jobs at different times and quick response is important.

Capacity Scheduler should be chosen when resource guarantees and organisational quotas are required, such as in large enterprises with separate business units. **Fair Scheduler** is a better choice for shared clusters where fairness, flexibility, and efficient resource utilisation are more important than fixed resource allocations.

5. Write the CLI command to:
 - (a) list all running applications

yarn application -list

- (b) kill a specific application

yarn application -kill <application_id>

- (c) retrieve logs for a completed application.

yarn logs -applicationId <application_id>

Explanation:

This command retrieves the execution logs for the specified application after it has completed. The logs can be used to review the application's output or troubleshoot errors. If

log aggregation is disabled or the logs have been removed, the command may report that no logs are available.

6. What happens when a container attempts to use more memory than it was allocated?

If a container uses more memory than it was allocated, **YARN kills the container**. This protects the cluster from running out of memory and ensures that resources are shared fairly among all applications. The ApplicationMaster can then restart the failed task in a new container if necessary.

7. What is data locality, and why does the RM scheduler try to honor it?

Data locality means **running tasks on the node where the data is stored, or on a nearby node**. The ResourceManager scheduler tries to honour data locality because it **reduces network traffic, improves performance, and allows jobs to finish faster** by avoiding unnecessary data transfers.

8. What command lets you change queue configuration without restarting the ResourceManager?

```
yarn rmadmin -refreshQueues
```

The command **yarn rmadmin -refreshQueues** reloads the queue configuration without restarting the ResourceManager. This allows administrators to update queue settings while keeping the YARN cluster running.

9. Fundamental difference between a Managed and an External table – what happens on DROP TABLE in each?

A **Managed Table** is controlled entirely by Hive, so when DROP TABLE is executed, both the table definition and its data are deleted. An **External Table** only stores the table definition in Hive, so DROP TABLE removes the metadata but leaves the actual data files unchanged.

10. Why is a low-cardinality partition key important? What goes wrong if you ignore that?

A **low-cardinality partition key** has only a few unique values, so it creates a reasonable number of partitions and improves query performance. If you ignore this and use a **high-cardinality** column, Hive creates too many partitions, making queries slower and increasing storage and metadata overhead.

11. Difference between static and dynamic partition – give a real scenario for each.

Static partitioning requires the user to specify the partition value before inserting data. For example, loading all January sales into the month='January' partition. **Dynamic partitioning** lets Hive create partitions automatically based on the data being loaded, such as creating separate partitions for each month when importing a full year's sales data.

12. You manually copied files into a new partition dir for an external table and queries return nothing. What command fixes it?

If you manually copy files into a new partition directory of an external table and Hive cannot see the data, run:

```
MSCK REPAIR TABLE table_name;
```

This command updates the Hive Metastore by discovering and registering the new partitions so they can be queried.

13. Compare bucketing and partitioning – when would you use one, the other or both?

Partitioning organises data into separate partitions based on a column such as **year** or **month**, allowing Hive to scan only the relevant data. **Bucketing** organises data into a fixed number of buckets using a hash function, which improves joins and query performance. Use **partitioning** for filtering data, **bucketing** for faster joins, and **both** together when working with very large datasets.

14. Why is ORC/Parquet preferred over TEXTFILE for analytical tables?

ORC and **Parquet** are preferred over **TEXTFILE** because they store data in a columnar format, allowing Hive to read only the columns needed for a query. They also provide better compression and faster query performance, making them ideal for large analytical tables.

15. Purpose of ANALYZE TABLE ...COMPUTE STATISTICS and its effect on query planning.

The **ANALYZE TABLE ... COMPUTE STATISTICS** command collects information about a table, such as its size and number of rows. Hive uses these statistics to optimise query planning by choosing more efficient execution plans, which improves query performance.

16. Write a query using a windows function to find each customer's top 3 highest-value orders.

```
SELECT customer_id,  
       order_id,  
       order_value  
FROM (  
  SELECT customer_id,  
         order_id,  
         order_value,  
         ROW_NUMBER() OVER (  
           PARTITION BY customer_id  
           ORDER BY order_value DESC  
         ) AS rank_num  
  FROM orders
```

```
) ranked_orders
```

```
WHERE rank_num <= 3;
```

This query uses the **ROW_NUMBER()** window function to rank each customer's orders from the **highest value to the lowest value**.

- PARTITION BY customer_id creates a separate ranking for each customer.
- ORDER BY order_value DESC sorts each customer's orders from highest to lowest value.
- ROW_NUMBER() assigns a rank starting from 1.
- WHERE rank_num <= 3 returns only the **top three highest-value orders** for each customer.

17. What core architectural decisions lets impala achieve much lower latency than Hive-on-MapReduce?

Impala achieves lower latency because it executes SQL queries directly using long-running processes and in-memory execution, without creating MapReduce jobs. Hive-on-MapReduce must start one or more MapReduce jobs for every query, which adds significant startup time and makes queries slower.

18. Explain the role of each daemon: impalad, statestored, catalogd.

? **impalad** executes SQL queries and processes the data.

? **catalogd** manages metadata and keeps it updated across the cluster.

? **statestored** monitors the health of the Impala cluster and helps coordinate communication between the daemons.

19. Difference between REFRESH and INVALIDATE METADATA – when do you use each?

Use **REFRESH** when the **data** in a table has changed, such as after adding new files. Use **INVALIDATE METADATA** when the **table structure or metadata** has changed outside Impala, such as after creating, dropping, or altering tables in Hive.

20. Why is COMPUTE STATES so important and what specifically goes wrong if skipped?

COMPUTE STATS collects statistics about a table so that Impala can optimise query execution. If it is skipped, Impala does not know enough about the data and may choose inefficient execution plans, resulting in slower queries and poorer overall performance.

21. Give a real scenario for choosing Hive over Impala and one for the reverse.

Use **Hive** when processing large amounts of data in scheduled batch jobs, such as generating daily sales reports overnight. Use **Impala** when users need fast, interactive queries, such as analysing live sales data or viewing business dashboards.

22. What happens to a running Impala query if a node fails mid-query? How does Hive differ?

If an **Impala** node fails during a query, the query usually fails and must be run again. In **Hive-on-MapReduce**, YARN automatically restarts the failed task on another node, so the job can continue. This makes Hive more fault tolerant, while Impala prioritises faster query execution.

23. Why is Parquet generally recommended for Impala-heavy workloads?

Parquet is recommended for Impala because it stores data by **columns**, allowing Impala to read only the required columns. It also provides better compression and faster query performance, making it ideal for interactive analytical workloads.

24. A colleague says “I loaded new data with Hive but Impala shows outdated results” – what do you tell them to run?

If Hive has loaded **new data**, tell your colleague to run:

```
REFRESH table_name;
```

If the **table structure or metadata** has changed, run:

```
INVALIDATE METADATA table_name;
```

REFRESH is used after data changes, while **INVALIDATE METADATA** is used after metadata or schema changes.

25. Why is Kafka a distributed replayable log, rather than a traditional queue, what does that unlock?

Kafka is a distributed replayable log because it stores messages even after they are consumed, unlike a traditional queue where messages are removed. This allows multiple consumers to read the same data, replay messages if needed, and provides better scalability, fault tolerance, and event streaming capabilities.

26. Difference between a topic and a partition and why partition count controls consumer parallelism.

A topic is a category where Kafka stores messages, while a partition is a smaller section of a topic that holds part of the data. The number of partitions controls consumer parallelism because each partition can be read by only one consumer within a consumer group at a time. More partitions allow more consumers to process messages simultaneously, improving performance and scalability.

27. What changes between a keyed and unkeyed message for partition assignment and ordering?

Keyed messages always go to the same partition, preserving order for that key. Unkeyed messages are spread across partitions for better load balancing, but ordering is not guaranteed across the topic.

28. What is a consumer group and how does Kafka ensure one partition = one member at a time?

A consumer group is a team of consumers reading a topic together. Kafka ensures that each partition is assigned to only one consumer in the group at a time and automatically rebalances partitions if consumers join or leave.

29. Describe at-most-once/ at-least-once/ exactly-once and how each is achieved.

At-most-once means a message is delivered zero or one time, so messages may be lost but are never duplicated. At-least-once guarantees every message is delivered, but some messages may be processed more than once. Exactly-once ensures each message is processed only once, with no duplicates or data loss, using Kafka's idempotent producers and transactions.

30. What is consumer lag, why does it matter and how do you keep it via CLI?

Consumer lag is the difference between the latest message in a Kafka partition and the last message a consumer has processed. It matters because high lag means consumers are falling behind and messages are waiting to be processed. You can monitor consumer lag from the command line using the kafka-consumer-groups CLI with the --describe option.

31. Relationship between acks=all, min, insync, replicas and durability guarantees.

acks=all waits for all in-sync replicas to confirm a write, while min.insync.replicas defines the minimum number of replicas required. Together, they improve data durability and fault tolerance.

32. A colleague's consumers group intermittently "loses" data after redplays – what would you check first?

I'd first check the consumer offsets and lag. Then I'd verify the offset reset policy and Kafka's retention settings, as incorrect offsets or expired messages can make it appear that data has been lost.

Conclusions

Through completing these exercises, I strengthened my understanding of the core components of the Hadoop ecosystem, including YARN, Hive, Impala, and Kafka. I learned how YARN manages cluster resources and applications, how Hive organises and optimises large datasets using partitioning, bucketing, and table statistics, and how Impala provides fast, interactive SQL queries through metadata management and query optimisation. I also gained an understanding of Kafka's architecture, including topics, partitions, consumer groups, message delivery guarantees, and durability features. Overall, these exercises improved both my theoretical knowledge and practical understanding of big data technologies, giving me greater confidence in working with distributed data processing systems and preparing me for real-world data engineering and analytics tasks.